CometBFT 합의 알고리즘 완전 분석

목차

1. CometBFT 개요
2. Byzantine Fault Tolerance 이론
3. Tendermint 합의 프로토콜
4. Consensus State Machine
5. Proposer Selection
6. Voting 메커니즘
7. Safety와 Liveness
8. 네트워크 파티션 대응
9. 실제 코드 분석
10. 성능 최적화

---

1. CometBFT 개요

1.1 CometBFT란?

CometBFT (구 Tendermint Core)는 **Byzantine Fault Tolerant** 합의 알고리즘을 구현한 범용 합의 엔진입니다.

주요 특징

- **즉시 Finality**: 블록이 커밋되면 절대 되돌릴 수 없음 (No reorg)
- **Byzantine Fault Tolerance**: 전체 노드의 1/3 미만이 악의적이어도 안전
- **ABCI 인터페이스**: 어떤 애플리케이션과도 연결 가능
- **고성능**: 수천 TPS 지원
- **단순성**: 복잡한 포크 선택 규칙 불필요

Bitcoin/Ethereum PoW vs CometBFT

항목	PoW (Bitcoin/Ethereum 1.0)	CometBFT
Finality	확률적 (6 confirmations)	즉시 Finality
Reorg 가능성	있음 (51% 공격)	없음
블록 시간	~10분 / ~12초	~1~7초 (설정 가능)
에너지	매우 높음	낮음
Validator	PoW miners	Staking validators
BFT 모델	No	Yes (최대 1/3 Byzantine)

1.2 아키텍처

```
┌──────────────────────────────────────────────┐
│                  Application                 │
│             (Cosmos SDK via ABCI)            │
└──────────────────────────────────────────────┘
                   │ ABCI Interface
┌──────────────────────────────────────────────┐
│                CometBFT Core                 │
│                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐   │
│  │ Consensus │  │ Mempool  │  │ P2P      │   │
│  │  Engine  │  │          │  │ Network  │   │
│  └──────────┘  └──────────┘  └──────────┘   │
│                                              │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐   │
│  │ State    │  │ Block Store │ │ Evidence │   │
│  │ Store    │  │          │  │ Pool     │   │
│  └──────────┘  └──────────┘  └──────────┘   │
└──────────────────────────────────────────────┘
```

---

2. Byzantine Fault Tolerance 이론

2.1 Byzantine Generals Problem

문제: 분산 시스템에서 일부 노드가 악의적으로 행동할 때, 정직한 노드들이 합의에 도달할 수 있는가?

```
General A (정직) ──────> "공격!" ──────> General B (정직)
                  \                    /
                   \                  /
                    \                /
                     \              /
                      General C (배신자)

C가 A에게는 "공격!"이라 말하고
C가 B에게는 "후퇴!"라고 거짓말하면?
→ A와 B의 합의 실패
```

2.2 BFT 안전성 정리

정리: `n` 개의 노드 중 최대 `f` 개가 Byzantine일 때, 안전한 합의를 위해서는:

```
n ≥ 3f + 1
```

즉, **전체의 2/3 + 1 이상이 정직**해야 합니다.

예시

- 총 노드 4개: 최대 1개 Byzantine 허용
- 총 노드 100개: 최대 33개 Byzantine 허용
- 총 노드 1000개: 최대 333개 Byzantine 허용

2.3 CometBFT의 BFT 구현

CometBFT는 **Practical BFT (PBFT)**를 개선한 **Tendermint BFT**를 사용합니다:

핵심 아이디어:

1. **Voting Rounds**: 여러 라운드의 투표를 통해 합의
2. **2/3+ Majority**: 각 단계에서 2/3 이상의 동의 필요
3. **Locking**: 한 번 투표하면 이전 결정에 Lock됨
4. **Timeout**: 진행이 멈추면 다음 라운드로

---

3. Tendermint 합의 프로토콜

3.1 합의 라운드 구조

각 블록 높이(height)마다 합의 라운드가 진행됩니다:

```
Height H, Round R:
  1. Propose (제안)
  2. Prevote (1차 투표)
  3. Precommit (2차 투표)
  4. Commit (확정)
```

3.2 상세 프로토콜

```
┌──────────────────────────────────────────────────┐
│            Height H, Round 0                     │
└──────────────────────────────────────────────────┘

Step 1: Propose
──────────────────────────────────────
Proposer (결정론적으로 선택됨):
  - Mempool에서 트랜잭션 선택
  - 블록 생성 및 네트워크에 브로드캐스트

Validators:
  - Proposal 수신
  - Proposal 유효성 검증 (ABCI ProcessProposal)

Timeout: TimeoutPropose (기본 3초)


Step 2: Prevote (1차 투표)
──────────────────────────────────────
각 Validator:
  IF (valid proposal 수신):
      Prevote(Proposal)  // "이 블록에 동의"
  ELSE:
      Prevote(nil)  // "유효한 제안 못 받음"

Network:
  - 모든 Prevote 메시지 브로드캐스트
  - 2/3+ Prevote 대기

Condition A: 2/3+ Prevote(Proposal) 받음
  → Proposal에 "Polka" 발생
  → LockedValue = Proposal
  → LockedRound = R

Condition B: 2/3+ Prevote(nil) 받음
  → 다음 라운드로

Timeout: TimeoutPrevote (기본 1초)


Step 3: Precommit (2차 투표)
──────────────────────────────────────
각 Validator:
  IF (2/3+ Prevote(Proposal) 받음):
      Precommit(Proposal)  // "이 블록 확정에 동의"
  ELSE:
      Precommit(nil)

Network:
  - 모든 Precommit 메시지 브로드캐스트

Condition A: 2/3+ Precommit(Proposal) 받음
  → Commit! (블록 확정)
  → Height H+1으로 진행

Condition B: 2/3+ Precommit(nil) 받음
  → Round R+1으로 진행

Timeout: TimeoutPrecommit (기본 1초)


Step 4: Commit
──────────────────────────────────────
  - ABCI FinalizeBlock(Proposal) 호출
  - ABCI Commit() 호출
  - 블록 영구 저장
  - Height H+1, Round 0으로 진행
```

3.3 시각화: 정상 케이스

```
Time →
──────────────────────────────────────────────────────────

Height 100, Round 0

0s    Propose      [Proposer] Block #100 제안
      ↓
      [Val1] Proposal 수신 ✓
      [Val2] Proposal 수신 ✓
      [Val3] Proposal 수신 ✓
      [Val4] Proposal 수신 ✓

1s    Prevote      [Val1] Prevote(Block #100)
      ↓            [Val2] Prevote(Block #100)
      ↓            [Val3] Prevote(Block #100)
      ↓            [Val4] Prevote(Block #100)
      ↓            → 4/4 = 100% > 2/3 ✓ Polka!

2s    Precommit    [Val1] Precommit(Block #100)
      ↓            [Val2] Precommit(Block #100)
      ↓            [Val3] Precommit(Block #100)
      ↓            [Val4] Precommit(Block #100)
      ↓            → 4/4 = 100% > 2/3 ✓ Commit!

3s    Commit       FinalizeBlock(Block #100)
      ↓            Commit()
      ↓            → Height 101, Round 0 시작
```

3.4 시각화: 네트워크 지연 케이스

```
Height 100, Round 0

0s    Propose      [Proposer] Block #100 제안
      ↓
      [Val1] Proposal 수신 ✓
      [Val2] Proposal 수신 ✓
      [Val3] Proposal 수신 ✗ (네트워크 지연)
      [Val4] Proposal 수신 ✗ (네트워크 지연)

1s    Prevote      [Val1] Prevote(Block #100)
      ↓            [Val2] Prevote(Block #100)
      ↓            [Val3] Prevote(nil)
      ↓            [Val4] Prevote(nil)
      ↓            → 2/4 = 50% < 2/3 ✗ No Polka

2s    Precommit    [All] Precommit(nil)
      ↓            → 100% Precommit(nil)
      ↓            → Round 1으로 진행

4s    Round 1      다시 시도 (Timeout 증가)
      Propose      [New Proposer] Block #100 제안
      ...
```

---

4. Consensus State Machine

4.1 State Machine 다이어그램

```
                NewHeight
                    ↓
            ┌──────────────┐
            │   NewRound   │
            └──────────────┘
                    │
                    ↓
            ┌──────────────┐
    ┌──────→│   Propose    │
    │       └──────────────┘
    │               │
    │               ↓
    │       ┌──────────────┐      Timeout
    │       │   Prevote    │──────────────┐
    │       └──────────────┘              │
    │               │                     │
    │               ↓                     │
    │       ┌──────────────┐      Timeout│
    │       │  Precommit   │──────────────┤
    │       └──────────────┘              │
    │               │                     │
    │               │                     │
    │               │ 2/3+ Precommit      │
    │               ↓                     │
    │       ┌──────────────┐              │
    │       │    Commit    │              │
    │       └──────────────┘              │
    │               │                     │
    │               ↓                     │
    │          NewHeight                  │
    │                                     │
    └─────────────────────────────────────┘
            NextRound
```

4.2 State 정의

```go
// cometbft/consensus/state.go
type RoundState struct {
    Height             int64               // 현재 블록 높이
    Round              int32               // 현재 라운드
    Step               RoundStepType       // Propose/Prevote/Precommit/Commit
    StartTime          time.Time           // 라운드 시작 시간
    CommitTime         time.Time           // Commit 시간
    Validators         *types.ValidatorSet // Validator 세트
    Proposal           *types.Proposal     // 현재 제안
    ProposalBlock      *types.Block        // 제안된 블록
    ProposalBlockParts *types.PartSet      // 블록 파트들
    LockedRound        int32               // Lock된 라운드
    LockedBlock        *types.Block        // Lock된 블록
    ValidRound         int32               // Valid 라운드
    ValidBlock         *types.Block        // Valid 블록
    Votes              *HeightVoteSet      // 투표 집합
    CommitRound        int32               // Commit 라운드
    LastCommit         *types.Commit       // 이전 Commit
    LastValidators     *types.ValidatorSet // 이전 Validators
}
```

4.3 Step 전환

```go
type RoundStepType uint8

const (
    RoundStepNewHeight     RoundStepType = 0x01 // Height 시작
    RoundStepNewRound      RoundStepType = 0x02 // Round 시작
    RoundStepPropose       RoundStepType = 0x03 // Proposal 대기
    RoundStepPrevoteWait   RoundStepType = 0x04 // Prevote 대기
    RoundStepPrecommitWait RoundStepType = 0x05 // Precommit 대기
    RoundStepCommit        RoundStepType = 0x06 // Commit 진행
)

// Step 전환 예시
func (cs *State) enterNewRound(height int64, round int32) {
    cs.updateRoundStep(round, RoundStepNewRound)
    cs.Validators = cs.Validators.Copy()
    cs.Proposal = nil
    cs.ProposalBlock = nil
    cs.ProposalBlockParts = nil
    cs.Votes.SetRound(round)

    // Proposer 선택
    cs.Proposer = cs.Validators.GetProposer()

    // Timeout 시작
    cs.scheduleTimeout(cs.config.Propose(round), height, round, RoundStepPropose)
}
```

---

5. Proposer Selection

5.1 Weighted Round-Robin 알고리즘

CometBFT는 **Voting Power에 비례**하여 Proposer를 선택합니다.

```go
// types/validator_set.go
func (vals *ValidatorSet) GetProposer() *Validator {
    if len(vals.Validators) == 0 {
        return nil
    }

    // Proposer는 incrementProposerPriority()로 결정
    return vals.Proposer
}

func (vals *ValidatorSet) IncrementProposerPriority(times int32) {
    for i := 0; i < int(times); i++ {
        vals.incrementProposerPriority()
    }
}

func (vals *ValidatorSet) incrementProposerPriority() {
    // 1. 모든 validator의 ProposerPriority += VotingPower
    for _, val := range vals.Validators {
        val.ProposerPriority += val.VotingPower
    }

    // 2. 가장 높은 ProposerPriority를 가진 validator가 Proposer
    proposer := vals.Validators[0]
    for _, val := range vals.Validators[1:] {
        if val.ProposerPriority > proposer.ProposerPriority {
            proposer = val
        }
    }

    // 3. 선택된 Proposer의 ProposerPriority -= TotalVotingPower
    proposer.ProposerPriority -= vals.TotalVotingPower()

    vals.Proposer = proposer
}
```

예시: 3명의 Validator

```
Initial State:
  Val A: VotingPower = 10, Priority = 0
  Val B: VotingPower = 20, Priority = 0
  Val C: VotingPower = 30, Priority = 0
  Total = 60

Round 1:
  Step 1: Add VotingPower to Priority
    A: Priority = 0 + 10 = 10
    B: Priority = 0 + 20 = 20
    C: Priority = 0 + 30 = 30 ← Highest

  Step 2: C becomes Proposer

  Step 3: Subtract TotalVotingPower from C
    C: Priority = 30 - 60 = -30

  Result: Proposer = C
  State: A=10, B=20, C=-30

Round 2:
  A: Priority = 10 + 10 = 20
  B: Priority = 20 + 20 = 40 ← Highest
  C: Priority = -30 + 30 = 0

  Proposer = B
  B: Priority = 40 - 60 = -20

  State: A=20, B=-20, C=0

Round 3:
  A: Priority = 20 + 10 = 30 ← Highest
  B: Priority = -20 + 20 = 0
  C: Priority = 0 + 30 = 30

  Proposer = A (또는 C, tie-break by address)
  A: Priority = 30 - 60 = -30

  State: A=-30, B=0, C=30
```

결과적으로 **VotingPower가 높을수록 더 자주** Proposer가 됩니다.

---

6. Voting 메커니즘

6.1 Vote 구조

```go
// types/vote.go
type Vote struct {
    Type             SignedMsgType  // Prevote or Precommit
    Height           int64          // 블록 높이
    Round            int32          // 라운드
    BlockID          BlockID        // 투표 대상 블록
    Timestamp        time.Time      // 투표 시간
    ValidatorAddress Address        // 투표자 주소
    ValidatorIndex   int32          // Validator 인덱스
    Signature        []byte         // 서명
}

type SignedMsgType byte

const (
    PrevoteType   SignedMsgType = 0x01
    PrecommitType SignedMsgType = 0x02
)
```

6.2 Vote 검증

```go
// consensus/state.go
func (cs *State) addVote(vote *types.Vote, peerID p2p.ID) error {
    // 1. 기본 검증
    if err := vote.ValidateBasic(); err != nil {
        return err
    }

    // 2. Height & Round 검증
    if vote.Height != cs.Height {
        return ErrInvalidHeight
    }

    // 3. Validator 확인
    valIndex := vote.ValidatorIndex
    if valIndex >= int32(len(cs.Validators.Validators)) {
        return ErrInvalidValidatorIndex
    }
    validator := cs.Validators.Validators[valIndex]

    // 4. 서명 검증
    if !validator.PubKey.VerifySignature(vote.SignBytes(cs.state.ChainID), vote.Signature) {
        return ErrInvalidSignature
    }

    // 5. VoteSet에 추가
    added, err := cs.Votes.AddVote(vote)
    if err != nil {
        return err
    }

    // 6. 2/3+ 도달 확인
    if added {
        if vote.Type == types.PrevoteType {
            cs.checkPrevoteThreshold(vote.Round, vote.BlockID)
        } else {
            cs.checkPrecommitThreshold(vote.Round, vote.BlockID)
        }
    }

    return nil
}
```

6.3 Vote Aggregation

```go
// types/vote_set.go
type VoteSet struct {
    chainID       string
    height        int64
    round         int32
    signedMsgType SignedMsgType
    valSet        *ValidatorSet

    votes            []*Vote            // Validator index → Vote
    votesBitArray    *bits.BitArray     // 투표 여부 비트맵
    votesByBlock     map[string]*blockVotes  // BlockID → 투표 집계
    peerMaj23        map[p2p.ID]BlockID // Peer가 본 2/3+ majority
}

// 2/3+ majority 확인
func (voteSet *VoteSet) TwoThirdsMajority() (blockID BlockID, ok bool) {
    if voteSet == nil {
        return BlockID{}, false
    }

    twoThirdsVotingPower := voteSet.valSet.TotalVotingPower() * 2 / 3

    for _, blockVotes := range voteSet.votesByBlock {
        if blockVotes.sum > twoThirdsVotingPower {
            return blockVotes.blockID, true
        }
    }

    return BlockID{}, false
}
```

---

7. Safety와 Liveness

7.1 Safety (안전성)

Safety: 정직한 validator들이 **서로 다른 블록을 Commit하지 않음**

증명 스케치

```
가정: Validator 세트의 2/3+가 정직

정리: Height H에서 두 개의 서로 다른 블록 B1, B2가 동시에 Commit될 수 없다.

증명:
  B1이 Commit되려면:
    - 2/3+ Precommit(B1) 필요

  B2가 Commit되려면:
    - 2/3+ Precommit(B2) 필요

  그러나:
    - 2/3+ Precommit(B1) ∩ 2/3+ Precommit(B2) ≠ ∅
    - 즉, 최소 1/3+의 validator가 B1과 B2 모두에 Precommit
    - Locking 규칙에 따라 validator는 한 번에 하나의 블록에만 Precommit 가능
    - 따라서 모순!

  ∴ B1 ≠ B2인 경우 둘 다 Commit 불가능
```

Locking 메커니즘

```
// consensus/state.go
func (cs *State) enterPrevote(height int64, round int32) {
    // Locked value가 있으면 그것에 Prevote
    if cs.LockedBlock != nil {
        cs.signAddVote(types.PrevoteType, cs.LockedBlock.Hash(), cs.LockedBlock.MakePartSet(cs.state.ConsensusParams.BlockPartSizeBytes
        return
    }

    // Proposal이 유효하면 Prevote
    if cs.ProposalBlock != nil && cs.isProposalComplete() {
        cs.signAddVote(types.PrevoteType, cs.ProposalBlock.Hash(), cs.ProposalBlockParts.Header())
        return
    }

    // 아니면 nil Prevote
    cs.signAddVote(types.PrevoteType, nil, types.PartSetHeader{})
}
```

7.2 Liveness (활성)

Liveness: 시스템이 **결국 진행**됨 (새로운 블록 생성)

문제: 무한 라운드

네트워크 파티션 또는 비동기 환경에서 합의가 멈출 수 있습니다.

해결: Exponential Backoff Timeout

```
// config/config.go
type ConsensusConfig struct {
    TimeoutPropose        time.Duration // 기본 3s
    TimeoutProposeDelta   time.Duration // 라운드당 증가량 500ms
    TimeoutPrevote        time.Duration // 기본 1s
    TimeoutPrevoteDelta   time.Duration // 라운드당 증가량 500ms
    TimeoutPrecommit      time.Duration // 기본 1s
    TimeoutPrecommitDelta time.Duration // 라운드당 증가량 500ms
    TimeoutCommit         time.Duration // 기본 1s (다음 높이로)
}

// 라운드가 증가할수록 Timeout 증가
func (cfg *ConsensusConfig) Propose(round int32) time.Duration {
    return time.Duration(
        cfg.TimeoutPropose.Nanoseconds()+cfg.TimeoutProposeDelta.Nanoseconds()*int64(round),
    ) * time.Nanosecond
}
```

효과:

- Round 0: 3초
- Round 1: 3.5초
- Round 2: 4초
- ...
- 네트워크가 복구되면 결국 합의 도달

---

8. 네트워크 파티션 대응

8.1 시나리오: 네트워크 분할

```
초기 상태: 4 Validators (각 25% voting power)

     [Val A]      [Val B]      [Val C]      [Val D]
        │            │            │            │
        └────────────┴────────────┴────────────┘
                   정상 통신

네트워크 파티션 발생:

     Partition 1          Partition 2
    ┌─────────┐          ┌─────────┐
    │ [Val A] │          │ [Val C] │
    │ [Val B] │          │ [Val D] │
    └─────────┘          └─────────┘
       50%                  50%
```

결과: 둘 다 2/3+ 도달 불가 → **합의 중단** (Safety 유지)

8.2 시나리오: 비대칭 파티션

```
3 Validators:
  Val A: 40% voting power
  Val B: 30% voting power
  Val C: 30% voting power

Partition:
  Group 1: [A, B] = 70% > 2/3 ✓
  Group 2: [C] = 30% < 2/3 ✗

결과:
  - Group 1: 합의 계속 진행 (새 블록 생성)
  - Group 2: 합의 중단 (블록 생성 안 됨)

파티션 복구 후:
  - C는 Group 1의 블록체인 따라잡기 (State Sync)
```

8.3 네트워크 복구

```go
// consensus/reactor.go
func (conR *Reactor) gossipVotesForHeight(height int64) {
    ps := conR.conS.GetRoundState()

    // 자신이 가진 Vote를 Peer에게 전송
    for _, peer := range conR.Switch.Peers().List() {
        peerState := peer.Get(types.PeerStateKey).(*PeerState)

        // Prevotes
        if prevotes := ps.Votes.Prevotes(ps.Round); prevotes != nil {
            for _, vote := range prevotes.Votes {
                if vote != nil && !peerState.HasVote(vote.Height, vote.Round, vote.Type, vote.ValidatorIndex) {
                    peer.Send(VoteChannel, MustEncode(&VoteMessage{Vote: vote}))
                }
            }
        }

        // Precommits
        if precommits := ps.Votes.Precommits(ps.Round); precommits != nil {
            for _, vote := range precommits.Votes {
                if vote != nil && !peerState.HasVote(vote.Height, vote.Round, vote.Type, vote.ValidatorIndex) {
                    peer.Send(VoteChannel, MustEncode(&VoteMessage{Vote: vote}))
                }
            }
        }
    }
}
```

9. 실제 코드 분석

9.1 Consensus Loop

```go
// consensus/state.go
func (cs *State) receiveRoutine(maxSteps int) {
    for {
        select {
        case <-cs.txNotifier.TxsAvailable():
            cs.handleTxsAvailable()

        case mi := <-cs.peerMsgQueue:
            cs.handleMsg(mi)

        case ti := <-cs.timeoutTicker.Chan():
            cs.handleTimeout(ti)

        case <-cs.Quit():
            return
        }
    }
}

// Timeout 처리
func (cs *State) handleTimeout(ti timeoutInfo) {
    cs.mtx.Lock()
    defer cs.mtx.Unlock()

    switch ti.Step {
    case cstypes.RoundStepPropose:
        // Proposal timeout → Prevote(nil)
        cs.enterPrevote(ti.Height, ti.Round)

    case cstypes.RoundStepPrevoteWait:
        // Prevote timeout → Precommit
        cs.enterPrecommit(ti.Height, ti.Round)

    case cstypes.RoundStepPrecommitWait:
        // Precommit timeout → NextRound
        cs.enterNewRound(ti.Height, ti.Round+1)
    }
}
```

9.2 Proposal 생성

```go
// consensus/state.go
func (cs *State) createProposalBlock() (*types.Block, error) {
    // ABCI PrepareProposal 호출
    txs := cs.mempool.ReapMaxBytesMaxGas(maxDataBytes, maxGas)

    prepareProposalReq := &abci.RequestPrepareProposal{
        MaxTxBytes:         maxDataBytes,
        Txs:                txs,
        LocalLastCommit:    cs.LastCommit.ToExtendedCommit(),
        Misbehavior:        cs.evpool.PendingEvidence(maxEvidence),
        Height:             cs.Height,
        Time:               cs.timestamp,
        NextValidatorsHash: cs.state.NextValidators.Hash(),
        ProposerAddress:    cs.Validators.GetProposer().Address,
    }

    prepareProposalResp, err := cs.proxyAppConn.PrepareProposal(cs.ctx, prepareProposalReq)
    if err != nil {
        return nil, err
    }

    // 블록 생성
    return cs.state.MakeBlock(
        cs.Height,
        prepareProposalResp.Txs,
        cs.LastCommit,
        cs.evpool.PendingEvidence(maxEvidence),
        cs.Validators.GetProposer().Address,
    )
}
```

9.3 Vote 서명

```go
// consensus/state.go
func (cs *State) signAddVote(msgType types.SignedMsgType, hash []byte, header types.PartSetHeader) *types.Vote {
    // Vote 생성
    vote := &types.Vote{
        Type:             msgType,
        Height:           cs.Height,
        Round:            cs.Round,
        BlockID:          types.BlockID{Hash: hash, PartSetHeader: header},
        Timestamp:        cs.voteTime(),
        ValidatorAddress: cs.privValidator.GetAddress(),
        ValidatorIndex:   cs.Validators.GetByAddress(cs.privValidator.GetAddress()),
    }

    // 서명
    signBytes := vote.SignBytes(cs.state.ChainID)
    signature, err := cs.privValidator.SignVote(cs.state.ChainID, vote)
    if err != nil {
        panic(err)
    }
    vote.Signature = signature

    // VoteSet에 추가
    cs.Votes.AddVote(vote)

    // 네트워크에 브로드캐스트
    cs.evsw.FireEvent(types.EventVote, vote)

    return vote
}
```

10. 성능 최적화

10.1 Block Gossip 최적화

문제: 큰 블록을 전송하면 네트워크 지연 발생

해결: Block Parts

```go
// types/part_set.go
const BlockPartSizeBytes = 65536 // 64 KB per part

// 블록을 여러 Part로 분할
func MakePartSet(partSize uint32, data []byte) *PartSet {
    numParts := (len(data) + int(partSize) - 1) / int(partSize)
    parts := make([]*Part, numParts)

    for i := 0; i < numParts; i++ {
        start := i * int(partSize)
        end := start + int(partSize)
        if end > len(data) {
            end = len(data)
        }

        parts[i] = &Part{
            Index: uint32(i),
            Bytes: data[start:end],
            Proof: merkle.ProofFromByteSlices(data, i),
        }
    }

    return &PartSet{
        total: uint32(numParts),
        parts: parts,
    }
}
```

효과:

- 병렬 전송 가능
- 일부 Part 누락 시 재전송만 하면 됨

10.2 Mempool 최적화

```go
// mempool/clist_mempool.go
type CListMempool struct {
    config        *cfg.MempoolConfig
    proxyMtx      sync.Mutex
    proxyAppConn  proxy.AppConnMempool
    txs           *clist.CList  // 트랜잭션 리스트
    txsMap        sync.Map      // Hash → *clistElement
    height        int64
    txsBytes      int64         // 총 메모리 사용량
    cache         TxCache       // 중복 방지
    preCheck      PreCheckFunc  // CheckTx 전 검사
    postCheck     PostCheckFunc // CheckTx 후 검사
}

// Reap: 블록에 포함할 트랜잭션 선택
func (mem *CListMempool) ReapMaxBytesMaxGas(maxBytes, maxGas int64) types.Txs {
    mem.updateMtx.RLock()
    defer mem.updateMtx.RUnlock()

    var (
        totalBytes int64
        totalGas   int64
        txs        []types.Tx
    )

    for e := mem.txs.Front(); e != nil; e = e.Next() {
        memTx := e.Value.(*mempoolTx)

        // Gas limit 체크
        if maxGas > -1 && totalGas+memTx.gasWanted > maxGas {
            continue
        }

        // Bytes limit 체크
        if maxBytes > -1 && totalBytes+int64(len(memTx.tx)) > maxBytes {
            break
        }

        totalBytes += int64(len(memTx.tx))
        totalGas += memTx.gasWanted
        txs = append(txs, memTx.tx)
    }

    return txs
}
```

10.3 Consensus Timeout 튜닝

```toml
# config.toml
[consensus]
timeout_propose = "3s"
timeout_propose_delta = "500ms"
timeout_prevote = "1s"
timeout_prevote_delta = "500ms"
timeout_precommit = "1s"
timeout_precommit_delta = "500ms"
timeout_commit = "1s"

# 빠른 네트워크 (예: 데이터센터)
timeout_propose = "1s"
timeout_prevote = "500ms"
timeout_precommit = "500ms"

# 느린 네트워크 (예: 글로벌)
timeout_propose = "5s"
timeout_prevote = "2s"
timeout_precommit = "2s"
```

참고 자료

논문

- The latest gossip on BFT consensus - Tendermint 논문
- Practical Byzantine Fault Tolerance - PBFT 원본 논문
- HotStuff: BFT Consensus in the Lens of Blockchain

공식 문서

- CometBFT Specification
- Consensus Algorithm Spec

코드

- CometBFT GitHub
- consensus/state.go

다음 단계

다음 문서에서는 **P2P 네트워크 및 Mempool**을 다룹니다:

- Gossip 프로토콜
- Peer discovery
- Mempool 관리
- Transaction propagation

→ 10-p2p-and-mempool.md